

System Architecture and Implementation Specification for an Automated Logic Synthesis, Minimization, and Visual Layout Backend Engine

Architectural System Overview and Runtime Foundations

The engineering design of a production-grade web backend for digital logic synthesis, optimization, and visualization requires a hybrid execution framework. The backend must ingest abstract logical specifications, reduce Boolean expressions using exact and heuristic minimization techniques, perform technology mapping to primitive target sets, emit Register-Transfer Level (RTL) code, and generate step-by-step graphical netlists. To fulfill these multi-stage processing requirements, the architecture relies on a runtime model that separates web request orchestration from high-performance symbolic algorithms. Abstract Syntax Tree (AST) construction, topological circuit graph traversal, and visual layout generation are executed within an asynchronous Node.js or Python application server. Heavy symbolic computation—including heuristic two-level logic minimization, Exclusive-OR Sum-Of-Products (ESOP) reductions, and hardware technology mapping—is offloaded to C/C++ libraries compiled to WebAssembly (WASM) or executed via native C extensions.

The engine operates across three distinct operational modes:

- **Boolean Expression Synthesis Mode:** Converts raw infix or prefix logic strings into ASTs, evaluates truth table representations, applies minimization algorithms, maps the reduced logic to target gate libraries (NAND, NOR, XOR), generates Verilog HDL, and produces rendered vector schematics.
- **Arbitrary Truth Table Ingestion Mode:** Processes variable-length truth tables directly, identifies active minterms and don't-care states, selects the appropriate exact or heuristic minimizer based on input dimensions, and routes the minimized expressions through the technology mapping and visual synthesis pipeline.
- **Circuit Schematic Extraction and Optimization Mode:** Ingests user-defined directed acyclic circuit netlist graphs, verifies topological stability, extracts symbolic Boolean expressions, evaluates circuit functionality to generate truth tables, and re-synthesizes optimized implementations.

Every phase of the pipeline generates deterministic, schema-validated JSON payloads. This isolation ensures that front-end clients can step backward and forward through the synthesis execution flow to render state visual snapshots at each phase.

Component Architecture and Technology Stack

The platform integrates specialized algorithmic engines to ensure scalable performance across varying input sizes. Small variable domains ($N \leq 6$) utilize exact solvers like Karnaugh

Maps, whereas larger variable domains ($N > 8$) dynamically engage heuristic tools like Berkeley Espresso.

Processing Layer	Engine / Primitive Core	Technology Stack	Operational Responsibility
Expression Parser	Nearley.js / ANTLR4 Engine	JavaScript / TypeScript	Tokenizes logic expressions, resolves operator precedence, and builds ASTs.
Bit-Parallel Truth Table	Vectorized Bitset Evaluator	C++ Engine / PyEDA Core	Evaluates and stores truth table bit vectors up to $N=32$ variables.
Visual K-Map Engine	Matrix Coordinate Mapper	TypeScript / Node.js	Computes 2D Gray Code layouts ($N \leq 6$) and prime implicant overlay geometries.
Exact Minimizer	Quine-McCluskey + Petrick Solver	C++ / Python Native Core	Calculates exact minimal sum-of-products for small variable spaces ($N \leq 8$).
Heuristic Minimizer	Berkeley Espresso C Core	C compiled to WASM / C Extension	Executes heuristic two-level minimization for multi-output logic.
XOR / ESOP Minimizer	EXORCISM-6 / Davio Engine	C++ / WebAssembly Core	Computes Exclusive-OR Sum-Of-Products expansions for parity logic.
Technology Mapper	Graph Rewriting Subsystem	TypeScript / Python	Maps logical expressions into target gate primitives (NAND, NOR, XOR).
Logic Synthesis Engine	Yosys Synthesis Framework	C++ compiled to yosys-wasm	Performs gate-level synthesis, cell mapping, and netlist optimization.
Schematic Layout Engine	NetlistSVG / ElkJS	Node.js / Browser Web Worker	Converts netlists into routed SVG schematics using force-directed layout algorithms.

Formal Data Schemas and Interface Contracts

System modules communicate via strictly typed JSON data structures. These contracts allow independent microservices or web-worker threads to process pipeline steps deterministically.

Abstract Syntax Tree Schema

The AST schema represents parsed logical expressions and structural sub-trees.

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "LogicASTNode",
  "type": "object",
  "properties": {
    "type": {
      "type": "string",
      "enum": ["OPERATOR", "VARIABLE", "CONSTANT"]
    },
    "operator": {
      "type": "string",
      "enum": ["NOT", "AND", "OR", "NAND", "NOR", "XOR", "XNOR",
"IMPLIES"]
    },
    "name": {
      "type": "string"
    },
    "value": {
      "type": "boolean"
    },
    "children": {
      "type": "array",
      "items": { "$ref": "#" }
    }
  },
  "required": ["type"]
}
```

Tri-State Truth Table Data Schema

The truth table payload handles multi-output systems and don't-care condition flags (X).

```
{
  "title": "TruthTablePayload",
  "type": "object",
  "properties": {
    "variables": {
      "type": "array",
      "items": { "type": "string" }
    },
    "outputs": {
      "type": "array",
      "items": { "type": "string" }
    }
  },
}
```

```

    "rows": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "inputs": {
            "type": "array",
            "items": { "type": "integer", "enum": [0, 1] }
          },
          "values": {
            "type": "array",
            "items": { "type": "string", "enum": ["0", "1", "X"] }
          }
        },
        "required": ["inputs", "values"]
      }
    },
    "required": ["variables", "outputs", "rows"]
  }
}

```

Unified Circuit Graph Netlist Schema

This format represents structural netlists, matching the standard yosys2digitaljs JSON interface.

```

{
  "title": "CircuitNetlist",
  "type": [span_47] (start_span) [span_47] (end_span) "object",
  "properties": {
    "modules": {
      "type": "object",
      "additionalProperties": {
        "ports": {
          "type": "object",
          "additionalProperties": {
            "direction": { "type": "string", "enum": ["input",
"output"] },
            "bits": { "type": "array", "items": { "type": "integer" } }
          }
        }
      },
      "cells": {
        "type": "object",
        "additionalProperties": {
          "type": { "type": "string" },
          "port_directions": {
            "type": "object",
            "additionalProperties": { "type": "string", "enum":

```

```
[ "input", "output" ] }
    },
    "connections": {
      "type": "object",
      "additionalProperties": {
        "type": "array",
        "items": { "type": "integer" }
      }
    }
  }
}
}
}
}
}
```

Algorithmic Logic Minimization and Transformation Engines

Expression Parsing and Vectorized Truth Table Generation

The parsing engine uses a custom Shunting-Yard algorithm to transform infix logic expressions into binary syntax trees. The parser enforces operator precedence in the following order: NOT, NAND, NOR, AND, XOR, XNOR, OR, and IMPLIES. Once built, the AST is evaluated across all variable combinations.

For truth tables with up to 64-bit words, variable assignments are mapped to CPU registers, allowing the engine to evaluate 64 input combinations in a single instruction pass.

Scalable Logic Minimization Suite

Visual Karnaugh Map Engine (Nile 6)

The visual K-Map sub-system translates truth table minterms into 2D matrices indexed by Gray Code ordering. The variables are partitioned into row variables (V_{row}) and column variables (V_{col}). Gray code functions $G(i) = i \oplus (i \gg 1)$ map binary indexes to grid positions:

$$\text{RowIndex}(m) = G^{-1}(m \gg \lceil V_{\text{col}} \rceil) \quad \text{ColIndex}(m) = G^{-1}(m \bmod 2^{\lceil V_{\text{col}} \rceil})$$

The grouping algorithm scans the grid for rectangular groupings of size $2^a \times 2^b$ containing active minterms (1) or don't-care cells (X). Prime implicants are calculated using Petrick's Method to select the minimum set of bounding boxes covering all active minterms.

Quine-McCluskey and Petrick Minimizer (Nile 8)

For exact sum-of-products minimization without grid rendering, the backend uses Quine-McCluskey:

1. Minterms are grouped by Hamming weight (number of set bits).
2. Adjacent groups are merged iteratively using the absorption law ($A \cdot B + A \cdot \bar{B} = A$), placing don't-care masks (-) in combined positions.
3. A prime implicant chart is constructed, and Petrick's algebraic method determines the absolute minimal cover set.

Berkeley Espresso Heuristic Core (N > 8)

When variable dimensions exceed 8 inputs, exact algorithms become computationally unfeasible due to NP-complete scaling. The backend dynamically delegates minimization tasks to a WASM port of the Berkeley Espresso library.

Espresso iteratively simplifies logic using three primary operations: * **EXPAND**: Expands implicants into maximal prime implicants by removing literals, checking against the OFF-set. * **IRREDUNDANT COVER**: Computes a minimal subset of prime implicants that covers the ON-set. * **REDUCE**: Shrinks prime implicants to smaller terms to escape local minima during subsequent expansion cycles.

ESOP and Reed-Muller Expansion Engine Functions containing heavy parity and XOR structures (such as arithmetic adders and error-correcting blocks) scale poorly under standard Sum-of-Products (SOP) forms. The backend routes these functions to an Exclusive-OR Sum-Of-Products (ESOP) engine.

This module utilizes Positive and Negative Davio Expansions to decompose logic trees into XOR chains:

$$\text{Positive Davio Expansion: } f = f_{\bar{x}_i} \oplus x_i \frac{\partial f}{\partial x_i}$$

$$\text{Negative Davio Expansion: } f = f_{x_i} \oplus \bar{x}_i \frac{\partial f}{\partial x_i}$$

Using the EXORCISM-6 heuristic framework, the engine minimizes the total number of XOR terms, reducing both transistor count and logic depth.

Universal Gate Technology Mapping

Once expressions are minimized, the mapping engine converts them into specific target cell libraries via dual-negation De Morgan transformations.

NAND-Only Technology Mapping NAND-only implementations rewrite operations using double negation ($\overline{\overline{Y}} = Y$) and De Morgan's identity ($\bar{A} + \bar{B} = \overline{A \cdot B}$):

- **NOT**: $\text{NOT}(A) \implies \text{NAND}(A, A)$
- **AND**: $\text{AND}(A, B) \implies \text{NAND}(\text{NAND}(A, B), \text{NAND}(A, B))$
- **OR**: $\text{OR}(A, B) \implies \text{NAND}(\text{NAND}(A, A), \text{NAND}(B, B))$
- **XOR**: $\text{XOR}(A, B) \implies \text{NAND}(\text{NAND}(A, \text{NAND}(A, B)), \text{NAND}(B, \text{NAND}(A, B)))$

NOR-Only Technology Mapping

NOR-only implementations apply the dual transform ($A \cdot B = \overline{\bar{A} + \bar{B}}$):

- **NOT**: $\text{NOT}(A) \implies \text{NOR}(A, A)$
- **OR**: $\text{OR}(A, B) \implies \text{NOR}(\text{NOR}(A, B), \text{NOR}(A, B))$
- **AND**: $\text{AND}(A, B) \implies \text{NOR}(\text{NOR}(A, A), \text{NOR}(B, B))$

- **XOR:** $\text{XOR}(A, B) \text{ implies } \text{NOR}(\text{NOR}(\text{NOR}(A, A), \text{NOR}(B, B)), \text{NOR}(A, B))$

XOR/XNOR ESOP Mapping

For ESOP expressions, logic trees map directly into balanced binary XOR tree structures. Cascading XOR chains reduce long delay paths by balancing operator depth across parallel execution pairs.

RTL Generation and Logic Synthesis

The structural logic tree is emitted as canonical Verilog-2001 source code. The code includes behavioral descriptions using assign statements and structural module instantiations. The Verilog output is compiled by yosys-wasm within a virtual file system. Yosys processes the design using target cell libraries to generate a gate-level JSON netlist.

```
# Yosys RTL Synthesis Mapping Script
read_verilog input_design.v
hierarchy -top top_module
proc; opt; fsm; opt
techmap; opt
abc -g NAND,NOR,XOR
clean
write_json output_netlist.json
```

Multi-Step Visual Layout Rendering

To visualize each transformation step, the system generates custom visual payloads at each stage:

1. **Step 1: Parse Visualization:** Renders an interactive AST hierarchy graph alongside a searchable truth table matrix.
2. **Step 2: Minimization Visualization:** Renders a 2D Karnaugh map grid complete with highlighted, color-coded prime implicant bounding overlays.
3. **Step 3: Technology Mapping Visualization:** Generates an intermediate gate-level schematic structure.
4. **Step 4: RTL Synthesis Visualization:** Runs NetlistSVG and ElkJS layout solvers on the Yosys JSON netlist to produce a fully routed vector schematic (SVG). ElkJS uses layered graph algorithms to minimize edge crossings and wire bends.

End-to-End Operational Workflows

Workflow 1: Logical Operation Input

The expression input workflow transforms human-readable Boolean expressions into mapped logic circuits and visual schematics.

When an expression string is received (e.g., $(A \text{ AND } B) \text{ OR } (\text{NOT } A \text{ AND } C)$), the engine parses the input into an AST and generates Visual_Step_1_AST JSON. The AST is then evaluated

across all variable combinations to build the truth table payload.

Next, the logic minimizer simplifies the equation (e.g., $f = A \cdot B + \bar{A} \cdot C$) and emits Visual_Step_2_KMap JSON containing grid dimensions and prime implicant bounding coordinates. The simplified expression is then converted into target gate primitives (such as NAND-only networks) using graph-rewriting rules.

Finally, the backend generates Verilog HDL code, compiles it using yosys-wasm, and renders the resulting netlist into an SVG schematic using NetlistSVG.

Workflow 2: Arbitrary Truth Table Ingestion

The truth table workflow synthesizes circuits directly from tabular definitions across arbitrary variable counts.

Upon receiving an input matrix containing variable names, input bit vectors, and output results, the engine separates minterms into active sets ($M_{\{ON\}}$) and don't-care sets ($M_{\{DC\}}$).

The system selects an optimal minimizer based on variable count:

- For $N \leq 8$: The Quine-McCluskey solver computes an exact minimal cover.
- For $N > 8$: The backend formats the matrix as a PLA file (.i, .o, .ilb, .ob, .p) and delegates processing to Berkeley Espresso via WASM.

The minimized output string is parsed back into an algebraic expression and passed through the technology mapper, Verilog generator, Yosys synthesis engine, and visual layout renderer.

Workflow 3: User Circuit Netlist Extraction and Re-Synthesis

The circuit extraction workflow ingests user-created schematics, extracts their symbolic Boolean functions, and re-synthesizes optimized implementations.

Algorithm 1: Circuit Topological Sort and Symbolic Logic Extraction

```
-----
-----
Input   : Circuit Graph DAG  $G = (V, E)$ 
Output  : Output Expressions Map, Primary Inputs Set

1: Initialize InDegree Map for all nodes
   [span_50] (start_span) [span_50] (end_span)  $v$  in  $V$  to 0
2: PrimaryInputs  $\leftarrow$  Empty Set
3: NodeExpressions  $\leftarrow$  Map()

4: for each edge  $(u, v)$  in  $E$  do
5:     InDegree[v]  $\leftarrow$  InDegree[v] + 1
6: end for

7: Queue  $Q \leftarrow$  Empty Queue
8: for each node  $v$  in  $V$  do
9:     if InDegree[v] == 0 then
10:         Push  $Q, v$ 
11:         if  $v.type == "INPUT\_PORT"$  then
12:             Add  $v.name$  to PrimaryInputs
13:             NodeExpressions[v]  $\leftarrow$  VariableNode( $v.name$ )
14:         end if
```



```

15:     end if
16: end for

17: while Q is not empty do
18:     u <- Pop Q
19:     for each outgoing neighbor v of u do
20:         InDegree[v] <- InDegree[v] - 1
21:         if InDegree[v] == 0 then
22:             Push Q, v
23:             InputsList <- GetIncomingExpressions(v, E,
NodeExpressions)
24:             NodeExpressions[v] <- BuildASTForGate(v.gateType,
InputsList)
25:         end if
26:     end for
27: end while

28: return NodeExpressions, PrimaryInputs

```

The extracted AST expressions are passed to the evaluation matrix to generate the circuit's truth table. The system then re-minimizes the extracted logic, applies gate mapping, generates clean Verilog, and renders an optimized visual schematic.

Production Backend Implementation Modules

Module A: Shunting-Yard AST Expression Parser Engine

```

import re

class ASTNode:
    def __init__(self, node_type, value=None, children=None):

self.[span_58](start_span)[span_58](end_span)[span_63](start_span)[span_63](end_span)
type = node_type # "OPERATOR", "VARIABLE", "CONSTANT"
self.value = value # "AND", "OR", "NOT", "XOR", "NAND", "NOR", var_name
self.children = children if children is not None else []

    def to_dict(self):
        return {
            "type": self.type,
            "value": self.value,
            "children": [c.to_dict() for c in self.children]
        }

def parse_boolean_expression(expression: str) -> ASTNode:
    token_pattern =

```

```

r'\s*([A-Za-z_][A-Za-z0-9_]*|!|&|\||\||\^|~|&|\||\(|\)|\)|\s*'
tokens = [t for t in re.findall(token_pattern, expression) if
t.strip()]

precedence = {'!': 4, '~': 4, 'NAND': 3, 'NOR': 3, '&': 2, '&&':
2, '^': 1, '|': 0, '||': 0}
right_associative = {'!': True, '~': True}

output_queue = []
operator_stack = []

def normalize_op(op):
    mapping = {'&&': 'AND', '&': 'AND', '||': 'OR', '|': 'OR',
'!': 'NOT', '~': 'NOT', '^': 'XOR'}
    return mapping.get(op, op)

for token in tokens:
    if re.match(r'^[A-Za-z_][A-Za-z0-9_]*$', token) and token not
in ('AND', 'OR', 'NOT', 'NAND', 'NOR', 'XOR'):
        output_queue.append(ASTNode("VARIABLE", value=token))
    elif token in precedence or token in ('AND', 'OR', 'NOT',
'NAND', 'NOR', 'XOR'):
        op = normalize_op(token)
        while (operator_stack and operator_stack[-1] != '(' and
precedence.get(operator_stack[-1], 0) >
precedence.get(op, 0) or
        (precedence.get(operator_stack[-1], 0) ==
precedence.get(op, 0) and not right_associative.get(op, False))):
            op_val = operator_stack.pop()
            _create_operator_node(op_val, output_queue)
            operator_stack.append(op)
        elif token == '(':
            operator_stack.append(token)
        elif token == ')':
            while operator_stack and operator_stack[-1] != '(':
                op_val = operator_stack.pop()
                _create_operator_node(op_val, output_queue)
            if operator_stack and operator_stack[-1] == '(':
                operator_stack.pop()

while operator_stack:
    op_val = operator_stack.pop()
    _create_operator_node(op_val, output_queue)

return output_queue[0]

def _create_operator_node(op_val, queue):
    if op_val == 'NOT':

```

```

        operand = queue.pop()
        queue.append(ASTNode("OPERATOR", value="NOT",
children=[operand]))
    else:
        right = queue.pop()
        left = queue.pop()
        queue.append(ASTNode("OPERATOR", value=op_val, children=[left,
right]))

```

Module B: WebAssembly Espresso Core Integration Bridge

```

import { loadEspressoWasmModule } from './espresso_wasm_wrapper';

export interface EspressoPLAInput {
  numInputs: number;
  numOutputs: number;
  inputNames: string[];
  outputNames: string[];
  type: 'f' | 'fd' | 'fr';
  coverMatrix: Array<{ input: string; output: string }>;
}

export class EspressoMinimizerBridge {
  private wasmModule: any;

  async initialize() {
    this.wasmModule = await loadEspressoWasmModule();
  }

  minimize(input: EspressoPLAInput): string[] {
    let plaText = `.i ${input.numInputs}\n`;
    plaText += `.o ${input.numOutputs}\n`;
    plaText += `.ilb ${input.inputNames.join(' ')}\n`;
    plaText += `.ob ${input.outputNames.join(' ')}\n`;
    plaText += `.type ${input.type}\n`;

    for (const row of input.coverMatrix) {
      plaText += `${row.input} ${row.output}\n`;
    }
    plaText += `.e\n`;

    const inputPtr = this.wasmModule.allocateUTF8(plaText);
    const resultPtr = this.wasmModule._run_espresso(inputPtr);
    const minimizedPLA = this.wasmModule.UTF8ToString(resultPtr);

    this.wasmModule._free(inputPtr);
    this.wasmModule._free(resultPtr);
  }
}

```

```

    return this.parsePLAResult(minimizedPLA);
}

private parsePLAResult(plaOutput: string): string[] {
    const lines = plaOutput.split('\n');
    const implicants: string[] = [];
    for (const line of lines) {
        if (line.startsWith('.') || line.startsWith('#') ||
!line.trim()) continue;
        const parts = line.trim().split(/\s+/);
        if (parts.length >= 2 && parts[1].includes('1')) {
            implicants.push(parts[0]);
        }
    }
    return implicants;
}
}

```

Module C: RTL Generation & Yosys WASM Synthesis API

```

const yosysWasm = require('yosys-wasm');
const netlistSvg = require('netlistgraph');

class RTLAndSynthesisEngine {
    generateStructuralVerilog(moduleName, inputs, outputs,
wireDeclarations, gateInstantiations) {
        let verilog = `module ${moduleName} (\n`;
        verilog += `    input wire ${inputs.join(', ')},\n`;
        verilog += `    output wire ${outputs.join(', ')}\n`;
        verilog += `);\n\n`;

        if (wireDeclarations.length > 0) {
            verilog += `    wire ${wireDeclarations.join(', ')};\n\n`;
        }

        for (const gate of gateInstantiations) {
            verilog += `    ${gate.type} ${gate.id}
(${gate.connections.join(', ')});\n`;
        }

        verilog += `\nendmodule\n`;
        return verilog;
    }

    async synthesizeToTargetGates(verilogCode, topModule) {
        const yosys = await yosysWasm.Init();
    }
}

```

```

yosys.write_file('input.v', verilogCode);

const script = `
  read_verilog input.v
  hierarchy -top ${topModule}
  proc; opt
  techmap; opt
  abc -g NAND,NOR,XOR
  clean
  write_json netlist.json
`;

yosys.run_script(script);
const netlistJsonStr = yosys.read_file('netlist.json');
return JSON.parse(netlistJsonStr);
}

renderSchematicSVG(netlistJson) {
  const svgLayoutEngine = new netlistSvg.Renderer();
  return svgLayoutEngine.render(netlistJson);
}
}

module.exports = RTLAndSynthesisEngine;

```

Edge Cases, Mitigation Protocols, and Formal Verification

Edge Case Management Protocols

System stability requires explicit handling of large variable spaces, incomplete specifications, cyclic graphs, and wrapping visual elements:

- **Large Variable Memory Scaling ($N > 16$):** Evaluating truth tables with more than 16 inputs creates 65,536 memory rows, scaling exponentially past 4 billion rows at $[\text{span_105}](\text{start_span})[\text{span_105}](\text{end_span})N = 32$. When input dimensions exceed 12 variables, the backend bypasses full truth table materialization. Logic is converted directly into Reduced Ordered Binary Decision Diagrams (ROBDDs) or And-Inverter Graphs (AIGs) for direct Espresso minimization.
- **Don't-Care Optimization Handling:** Incompletely specified functions with active don't-care states (X) must maintain explicit ON-set (F) and DON'T-CARE (D) covers. The OFF-set (R) is left implicit ($R = \overline{F \cup D}$) to prevent invalid cell assignment during universal gate mapping.
- **Cyclic Netlist Routing:** User-submitted circuit graphs may contain feedback loops (such as ring oscillators or latches). The circuit parser runs Tarjan's Strongly Connected Components algorithm prior to logic extraction. If a cycle is detected, the engine raises an

error and highlights the feedback nets on the schematic canvas.

- **Visual K-Map Edge Wrapping:** Prime implicant groups spanning grid boundaries (e.g., four-corner groupings) require split render geometries. The matrix generator calculates split bounding arrays tagged with coordinate flags (wrapX: true, wrapY: true), enabling the UI layer to render split corner highlights accurately.

Synthesis Engine Verification Pipeline

To guarantee functional correctness across all optimization stages, synthesized output designs are verified against their original specifications.

Verification Pass	Target Stage	Algorithmic Mechanism	Success Criterion
BDD Equivalence Checking	Minimized SOP vs. Input AST	Constructs canonical Reduced Ordered Binary Decision Diagrams.	Identical pointer-isomorphic ROBDD root nodes.
SAT Miter Equivalence	Synthesized Gate Netlist vs. AST	Connects outputs to an XOR gate miter ($E = C_{\{src\}} \oplus C_{\{syn\}}$) and runs PicoSAT.	Solver returns UNSATISFIABLE (no failing input vectors exist).
Bit-Level Simulation Verification	RTL Output vs. Ingested Table	Simulates Verilog output across vector samples using bitwise evaluation engines.	Output patterns match ingested truth table values across all rows.

Works cited

1. Espresso heuristic logic minimizer - Wikipedia, https://en.wikipedia.org/wiki/Espresso_heuristic_logic_minimizer 2. Technology Mapping and Optimization Algorithms for Logic Synthesis of Advanced Technologies - The IC Web Server Home Page is in:ic.epfl.ch, <https://si2.epfl.ch/demichel/graduates/theses/alessandro.pdf> 3. package.json - nturley/netlistsvg - GitHub, <https://github.com/nturley/netlistsvg/blob/master/package.json> 4. DigitalJS Online, <https://digitaljs.tilk.eu/> 5. pyeda.boolalg.minimization — Logic Minimization — Python EDA Documentation, <https://pyeda.readthedocs.io/en/latest/reference/boolalg/minimization.html> 6. pyeda · PyPI, <https://pypi.org/project/pyeda/> 7. Boolean function optimizer package for Python - Stack Overflow, <https://stackoverflow.com/questions/5922315/boolean-function-optimizer-package-for-python> 8. A modern (2017) compilable re-host of the Espresso heuristic logic minimizer. - GitHub, <https://github.com/classabbyamp/espresso-logic> 9. Boolean function minimization for more than 100 variables, <https://electronics.stackexchange.com/questions/270732/boolean-function-minimization-for-more-than-100-variables> 10. Hardware Studio - GitHub Pages, <https://umarcor.github.io/hwstudio/doc/> 11. Two-level Logic Minimization — Python EDA Documentation - PyEDA, <https://pyeda.readthedocs.io/en/latest/2llm.html> 12. BE Information Technology (Four-Year Full Time) DEGREE PROGRAMME - Choice Based Credit System - Annamalai University, [https://annamalaiuniversity.ac.in/studport/download/Handbook/2022-23/IT/B.E_Information_Technology_Syllabus_\(Regulations-2022\).pdf](https://annamalaiuniversity.ac.in/studport/download/Handbook/2022-23/IT/B.E_Information_Technology_Syllabus_(Regulations-2022).pdf) 13. Data Encoding, Compilation, and Algorithms for

Quantum Machine Learning - s2.SMU, https://s2.smu.edu/~mitch/ftp_dir/pubs/Sinha25.pdf